

Técnicas Probabilísticas I

Sampling y Reglas de Asociación

IIC2440 — Procesamiento de Datos Masivos

Clase 05 — 2026-1

Juan Reutter

Agenda

1. **Bloque A** — Motivación: de determinista a probabilístico (~15 min)
2. **Bloque B** — Reglas de asociación (~20 min)
3. **Bloque C** — Apriori, comprimido (~15 min)
4. **Bloque D** — Sampling al rescate (~20 min)
5. — *break* —
6. **Bloque E** — Algoritmo de Toivonen (~20 min)
7. **Bloque F** — Sampling en streams: Reservoir (~15 min)
8. **Cierre y adelanto** (~5 min)

Bloque A

Motivación: de determinista a probabilístico

Hasta ahora

Para trabajar con datos masivos hemos usado:

- **Índices** para saltar directo a los datos relevantes
- **Almacenamiento columnar** para leer solo lo que importa
- **Particionamiento / distribución** para paralelizar

Todo esto asume algo muy fuerte:

Que con suficiente ingeniería, podemos seguir computando **la respuesta exacta** de forma eficiente.

El problema

A veces los datos son tan masivos que esta suposición se rompe:

- Un stream de eventos del que solo podemos almacenar el 1%
- Billones de pares posibles que no podemos comparar todos
- Una tabla tan grande que no cabe en RAM, ni siquiera particionada

En estos casos, **calcular la respuesta exacta es imposible**, aun con todos los trucos del mundo.

El giro de esta unidad

Vamos a abandonar la exactitud a cambio de velocidad. Dos técnicas:

- **Sampling** — trabajar con un subconjunto de los datos
- **Hashing** — transformar los datos a un espacio más pequeño

Ambas son **probabilísticas**: con cierta probabilidad, la respuesta puede ser incorrecta.

El arte consiste en diseñar la estructura para que esa probabilidad sea **tan baja como necesitamos**, y que el trade-off valga la pena.

Mapa de la unidad

Clase	Técnica	Aplicación
05 (hoy)	Sampling	Reglas de asociación (Apriori, Toivonen) + streams
06	Hashing	Bloom filters, Count-Min, HyperLogLog
07	LSH	Búsqueda de similitud, bases de datos de vectores

Todas comparten la misma filosofía: **perder un poco de exactitud para ganar mucho en velocidad.**

Bloque B

Reglas de asociación

El escenario clásico

Un supermercado registra cada compra como un conjunto de productos (una **canasta**):

```
C1 = {pan, leche, huevos}  
C2 = {pañales, cerveza, papel}  
C3 = {pan, mantequilla, leche}  
C4 = {pañales, cerveza, snacks}  
...
```

Queremos descubrir **regularidades**: qué productos se compran juntos, qué lleva a qué.

Reglas de asociación

Una regla de asociación tiene la forma:

$$I \rightarrow j$$

donde $I \subseteq E$ es un conjunto de elementos y $j \in E$ es un elemento.

Interpretación: en las canastas que contienen todos los elementos de I , es probable que también aparezca j .

El ejemplo clásico:

$$\text{pañales} \rightarrow \text{cerveza}$$

Soporte y confianza

Dadas canastas $C_1, \dots, C_n$:

Soporte de un conjunto S :

$$\text{sop}(S) = |\{C_i \mid S \subseteq C_i\}|$$

Cuántas canastas contienen a todos los elementos de S .

Confianza de la regla $I \rightarrow j$:

$$\text{conf}(I \rightarrow j) = \frac{\text{sop}(I \cup \{j\})}{\text{sop}(I)}$$

Qué proporción de las canastas que tienen I también tienen j .

El problema formal

Dado un conjunto E de elementos, canastas $C_1, \dots, C_n$, y umbrales $s, c \in [0, 1]$:

Encontrar **todas** las reglas $I \rightarrow j$ con soporte $\geq sn$ y confianza $\geq c$.

- Soporte alto $\rightarrow$ la regla es **frecuente** (no es una rareza)
- Confianza alta $\rightarrow$ la regla es **fiable** (cuando pasa I , pasa j)

Ambos umbrales importan: una regla con confianza 1 pero soporte 2 no es interesante.

Bloque C

Apriori (comprimido)

Apriori: la idea por niveles

Minar reglas por tamaños crecientes de I :

1. **Nivel 1**: calcular soporte de cada elemento individual. Descartar los que no alcanzan sn .
2. **Nivel $\ell+1$** : solo considerar conjuntos de tamaño $\ell+1$ formados **combinando sobrevivientes** del nivel ℓ . Descartar los que no alcanzan sn .
3. **Termina** cuando un nivel no produce nuevos conjuntos.
4. **Output**: reglas $I \rightarrow j$ con soporte y confianza suficientes.

Correcto y completo.

Por qué funciona, y por qué es lento

La propiedad anti-monótona (la clave):

Si S no alcanza soporte sn , ningún $S' \supseteq S$ lo alcanzará.

Apriori nunca calcula soporte para un superconjunto de algo ya descartado. Ese es todo el ahorro.

Pero sigue siendo lento: cada nivel ℓ requiere **una pasada completa** por las n canastas. Con $n = 10^8$ y reglas que viven en niveles 3-4, son decenas de pasadas sobre todo el dataset.

¿Y si cada pasada la hacemos sobre n/k canastas en vez de n ?

Mini-ejemplo

$C_1 = \{a, b, c\}$
 $C_2 = \{a, b, d\}$
 $C_3 = \{a, c, d\}$
 $C_4 = \{a, b, c\}$
 $C_5 = \{c, e\}$

Nivel 1 (soporte ≥ 2 de 5 canastas):

$\text{sop}(a)=4, \text{sop}(b)=3, \text{sop}(c)=4, \text{sop}(d)=2 \checkmark$

$\text{sop}(e)=1 \times \rightarrow$ descartado

Nivel 2 (combinando sobrevivientes):

$\text{sop}(\{a,b\})=3, \text{sop}(\{a,c\})=3, \text{sop}(\{a,d\})=2,$

$\text{sop}(\{b,c\})=2, \text{sop}(\{c,d\})=1 \times, \text{sop}(\{b,d\})=1 \times$

Nivel 3:

$\text{sop}(\{a,b,c\})=2 \checkmark$

$\{a,b,d\}$ descartado (contiene $\{b,d\}$)

$\{a,c,d\}$ descartado (contiene $\{c,d\}$)

La propiedad anti-monótona podó dos candidatos en el nivel 3 **sin mirarlos**.

Bloque D

Sampling al rescate

La idea

Apriori es lento porque cada nivel recorre las n canastas completas.

Estrategia simple: tomar un sample aleatorio de n/k canastas y correr Apriori sobre el sample.

Con $n = 10^8$ y $k = 100$, trabajamos con 10^6 canastas — **100× más rápido.**

Ojo con el umbral: si antes buscábamos soporte $\geq sn$, en el sample debemos buscar $\geq sn/k$ (mismo s proporcional, pero sobre menos canastas).

Errores que aparecen

El sample puede engañarnos de dos formas:

- **Falsos positivos:** reglas que alcanzan el soporte en el sample pero **no** en el dataset completo. *Aparecen en el output, no deberían.*
- **Falsos negativos:** reglas que **no** alcanzan el soporte en el sample pero sí en el dataset completo. *No aparecen en el output, deberían.*

Ejercicio (en vivo)

¿Se pueden eliminar los falsos positivos con operaciones posteriores? ¿Y los falsos negativos?

Si no se pueden eliminar, ¿cómo los mitigamos?

Pensemos 2 minutos antes de discutir.

Respuesta

Falsos positivos:  se pueden eliminar.

- Para cada regla candidata del sample, damos una pasada por el dataset completo verificando el soporte real. Si no lo alcanza, la descartamos.
- Costo: una pasada extra, pero solo sobre las reglas candidatas.

Falsos negativos:  no se pueden eliminar sin repensar el algoritmo.

- Si una regla nunca apareció en el sample, nunca la consideramos.
- Mitigación: bajar el umbral de soporte en el sample a psn/k con $p < 1$ (más reglas candidatas → menos falsos negativos, pero más trabajo).

Bloque E

Algoritmo de Toivonen

La promesa

Toivonen trabaja con un sample de n/k canastas, como antes, pero:

- **No produce falsos positivos ni falsos negativos.**
- A cambio: una pasada adicional por el dataset completo, y una **pequeña probabilidad de tener que reintentar.**

La idea clave: calcular dos "bordes" a partir del sample y verificarlos en el dataset completo.

Borde positivo del sample

Con umbral s en el dataset completo (buscamos soporte $\geq sn$):

El **borde positivo** son todos los conjuntos que alcanzan soporte $\geq psn/k$ en el sample, para algún $p < 1$.

Bajamos el umbral (con p) para **sobre-generar candidatos**: preferimos arrastrar conjuntos de más que perder alguno frecuente.

Intuición: si una regla es frecuente en el dataset completo, con alta probabilidad alcanza el umbral rebajado en el sample.

Borde negativo del sample

El **borde negativo** son los conjuntos que **no** están en el borde positivo, pero **todos** sus subconjuntos inmediatos sí lo están.

Son los "recién descartados": los candidatos que justo no alcanzaron el umbral rebajado, pero que estuvieron cerca.

Ejemplo: si $\{a, b\}$, $\{a, c\}$, $\{b, c\}$ están en el borde positivo, pero $\{a, b, c\}$ no, entonces $\{a, b, c\}$ está en el borde negativo.

La pasada final

Ahora damos **una pasada por el dataset completo** calculando el soporte real de todos los elementos en ambos bordes.

Caso 1 — Éxito: Ningún conjunto del borde negativo alcanza soporte sn .

- Output: las reglas del borde positivo que sí alcanzan sn en el dataset completo.
- **Sin falsos positivos ni negativos.**

Caso 2 — Fracaso: Algún conjunto del borde negativo sí alcanza soporte sn .

- No podemos confiar en el output.
- **Repetir con otro sample aleatorio.**

¿Por qué funciona?

No hay falsos positivos: verificamos el soporte real antes de emitir.

No hay falsos negativos en el caso de éxito. Supongamos que S es frecuente en el dataset completo ($\text{sop}(S) \geq sn$) pero **no** está en el borde positivo. Queremos mostrar que el algoritmo **detecta el problema**.

Tomemos el subconjunto $S' \subseteq S$ **más pequeño** que no está en el borde positivo:

- Todo subconjunto propio de S' **sí** está en el borde positivo (por minimalidad).
- Por definición, S' está entonces en el **borde negativo**.
- Por anti-monotonía, $\text{sop}(S') \geq \text{sop}(S) \geq sn$ en el dataset completo.

→ Algún elemento del borde negativo alcanza soporte $sn \rightarrow$ **caso 2, reintentar.** ■

Bloque F

Sampling en streams

El problema

Hasta ahora asumimos que el dataset está **guardado en disco** y podemos muestrear antes de empezar.

¿Y si los datos vienen como un **stream infinito**?

- Eventos de una web, logs de un servidor, tweets
- No sabemos n (o es efectivamente infinito)
- No podemos guardar todo para muestrear después

Queremos mantener un sample de tamaño fijo K tal que, en cualquier momento, cada elemento visto hasta ahora tenga **la misma probabilidad de estar en el sample**.

Reservoir sampling — Algoritmo R

```
reservoir = []
for i, x in enumerate(stream):
    if i < K:
        reservoir.append(x)           # llenar los primeros K
    else:
        j = random.randint(0, i)     # j ∈ [0, i]
        if j < K:
            reservoir[j] = x          # reemplazar con probabilidad K/(i+1)
```

Una pasada, memoria $O(K)$, no necesita conocer n .

Corrección — enunciado

Afirmación. Después de procesar n elementos $x_1, \dots, x_n$ (con $n \geq K$), para todo $i \leq n$:

$$P(x_i \in \text{reservoir}) = \frac{K}{n}$$

Es decir, **cada elemento visto tiene la misma probabilidad** de estar en el sample, independiente de cuándo llegó.

Esto es lo que queremos: un sample **uniforme** sobre todo el stream visto hasta ahora, mantenido en una sola pasada y con memoria $O(K)$.

Corrección — probabilidad de sobrevivir un paso

Supongamos que x_i está en el reservoir, y llega el elemento x_j con $j > K$. ¿Cuál es la probabilidad de que x_i **signa** en el reservoir tras este paso?

El nuevo elemento x_j entra con probabilidad K/j . Si entra, reemplaza **uniformemente al azar** uno de los K slots. Entonces:

$$P(x_i \text{ es reemplazado en paso } j) = \frac{K}{j} \cdot \frac{1}{K} = \frac{1}{j}$$

$$P(x_i \text{ sobrevive paso } j) = 1 - \frac{1}{j} = \frac{j-1}{j}$$

Los pasos son independientes (cada uno usa un sorteo fresco), así que podemos multiplicar.

Corrección — prueba (caso $i > K$)

x_i entra al reservoir con probabilidad K/i , luego debe sobrevivir los pasos $i+1, \dots, n$:

$$P(x_i \in \text{reservoir}) = \frac{K}{i} \cdot \prod_{j=i+1}^n \frac{j-1}{j}$$

El producto **telescopia**:

$$\prod_{j=i+1}^n \frac{j-1}{j} = \frac{i}{i+1} \cdot \frac{i+1}{i+2} \cdots \frac{n-1}{n} = \frac{i}{n}$$

$$\text{Por lo tanto } P(x_i \in \text{reservoir}) = \frac{K}{i} \cdot \frac{i}{n} = \frac{K}{n}.$$

Corrección — prueba (caso $i \leq K$)

x_i entra al reservoir con probabilidad 1, y debe sobrevivir los pasos $K+1, \dots, n$:

$$P(x_i \in \text{reservoir}) = 1 \cdot \prod_{j=K+1}^n \frac{j-1}{j} = \frac{K}{n} \quad \blacksquare$$

El mismo argumento telescópico, pero arrancando desde $K+1$ en vez de $i+1$.

En ambos casos llegamos a K/n : **cualquier elemento visto tiene la misma probabilidad** de estar en el sample, sin importar cuándo llegó.

Demo

```
import random

def reservoir_sample(stream, K):
    reservoir = []
    for i, x in enumerate(stream):
        if i < K:
            reservoir.append(x)
        else:
            j = random.randint(0, i)
            if j < K:
                reservoir[j] = x
    return reservoir

sample = reservoir_sample(range(1_000_000), K=10)
```

Repitiendo muchas veces, cada posición del stream aparece con frecuencia uniforme.

Cierre

Lo que vimos hoy

Sampling offline (dataset estático):

- Apriori es correcto pero lento — múltiples pasadas completas.
- Sampling ingenuo acelera pero introduce falsos positivos y negativos.
- **Toivonen** elimina ambos errores a costa de una pasada extra y una probabilidad de reintentar.

Sampling online (stream):

- **Reservoir sampling** mantiene un sample uniforme de tamaño fijo en una sola pasada.

Tema común: cambiar **exactitud garantizada** por **velocidad + probabilidad controlada de error**.

Próxima clase

Hashing para estadísticas sobre streams:

- **Bloom filters** — "¿este elemento está en un conjunto?" con memoria fija
- **Count-Min Sketch** — frecuencias aproximadas
- **HyperLogLog** — conteo de distintos con memoria constante

Todas siguen la misma filosofía de hoy, pero usando funciones de hash en vez de sampling.

Fin

¿Preguntas?